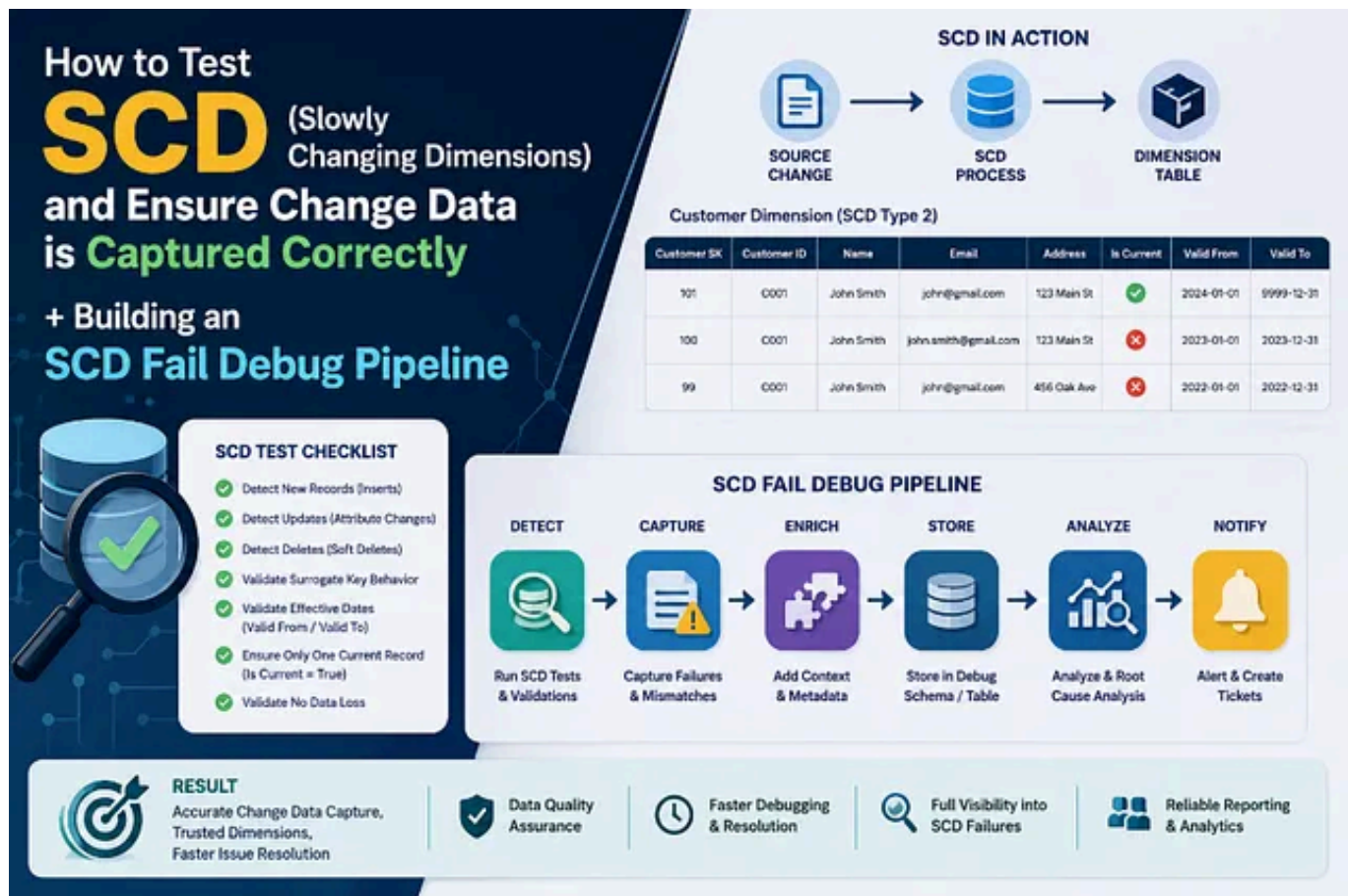




How to Test SCD (Slowly Changing Dimensions) and Ensure Change Data is Captured Correctly + Building an SCD Fail Debug Pipeline



Anurag Sharma · 5 min read · 2 days ago



Slowly Changing Dimensions (SCD) are a foundational concept in data warehousing and dimensional modeling. They allow us to track historical changes in dimension attributes (e.g., customer address, product category, employee title) without losing the ability to analyze data as it existed at any

Open in app ↗



Medium



Search



Write



to broken historical reporting, duplicate records, gaps in history, or performance issues. This detailed guide covers testing strategies for common SCD types (focusing on Type 2, the most popular for history) and how to build a robust debug/failure-handling pipeline.

Quick Recap: SCD Types

- **Type 0:** Never changes (e.g., birth date).
- **Type 1:** Overwrite (no history).
- **Type 2:** Add new row with effective/expiry dates + current flag (preserves full history).
- **Type 3:** Add old value column (limited history).
- **Type 4/6/Hybrid:** More advanced (mini-dimensions, etc.).

Type 2 is the focus here because it is complex to implement and test correctly.

Why Testing SCD Matters

Common failures include:

- Missing history (changes overwritten).
- Overlapping validity periods.
- Duplicate current records.
- Incorrect effective/expiry dates.
- Change detection missing subtle updates (e.g., null handling, trimming).
- Performance degradation on large volumes.

Step-by-Step: How to Test SCD Implementations

1. Setup Test Data and Scenarios

Create controlled test cases covering:

- New records (inserts).
- No-change records (should be ignored or handled as current).
- Attribute changes (Type 2: expire old, insert new).
- Multiple changes in one load.
- Deletes (soft/hard, depending on requirements).
- Edge cases: Nulls, special characters, date boundaries, high-volume batches, late-arriving data.

Use a source table and staging area to simulate changes.

2. Core Validation Tests for SCD Type 2

Here are key tests (adaptable to SQL, Spark, dbt, etc.):

Test A: Current Records Match Source (This should return zero rows.)

```

-- Compare current dim records (IsCurrent = TRUE) with latest source
SELECT
    s.business_key,
    s.attr1, s.attr2,
    d.attr1 as dim_attr1, d.attr2 as dim_attr2
FROM source_latest s
LEFT JOIN dim_table d
    ON s.business_key = d.business_key
    AND d.is_current = TRUE
WHERE s.attr1 != d.attr1
    OR s.attr2 != d.attr2
    OR d.business_key IS NULL;

```

Test B: No Overlapping or Gaps in History

```

--Check for overlaps
SELECT business_key, COUNT(*)
FROM dim_table
GROUP BY business_key, effective_date, expiry_date
HAVING COUNT(*) > 1;

--Check contiguous history(for each key,next effective=previous expiry + 1 day)
WITH ordered AS (
    SELECT business_key, effective_date, expiry_date,
           LEAD(effective_date)
             OVER (PARTITION BY business_key
                   ORDER BY effective_date) as next_eff
    FROM dim_table
)
SELECT * FROM ordered
WHERE next_eff IS NOT NULL
    AND next_eff != DATEADD(day, 1, expiry_date);

```

Test C: Uniqueness and Integrity

- Natural key + surrogate key uniqueness.

- Only one IsCurrent = TRUE per business key.
- Effective date <= Expiry date (or NULL for current).
- No duplicates on the full attribute set for the same validity period.

Test D: Historical Query Validation Simulate point-in-time queries:

```
SELECT * FROM dim_table
WHERE business_key = '<PK/BK>'
AND <UpdateDateColumn> BETWEEN effective_date AND COALESCE(expiry_date, CURRENT_TIMESTAMP)
```

Compare results against expected manual snapshots.



Test E: Change Detection Accuracy Use hashing for robust comparison:

```
-- In staging or pipeline
MD5(CONCAT(attr1, '|', attr2, '|', ...)) AS data_hash
```

Compare hashes between source and current dim to detect real changes (ignore metadata fields).

Additional tests:

- Performance under load.
- Idempotency (re-running the load with the same data doesn't create duplicates).
- Null/trim/whitespace handling.

3. Tools for Automated Testing

- ETL Validator or similar for rule-based checks (duplicate, query compare).
- Great Expectations or DBT tests for assertions.
- Unit tests in PySpark/SQL with mocked data.
- Data quality frameworks to run these after every load.

Building an SCD Fail Debug Pipeline

When SCD processing fails (or produces bad data silently), you need quick diagnosis and recovery.

Here's a practical debug pipeline design (adaptable to Airflow, Dagster, Spark, SQL, etc.).

Architecture Overview

1. **Logging & Monitoring Layer** — Capture everything.
2. **Validation Layer** — Run tests post-load; fail fast or alert.
3. **Debug/Quarantine Layer** — Isolate bad records.
4. **Recovery Layer** — Easy reprocessing.
5. **Alerting & Dashboarding.**

Example Implementation (PySpark + Delta Lake Style)

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import *
import logging
from datetime import datetime

spark = SparkSession.builder.appName("SCD_Debug_Pipeline").getOrCreate()

# Setup logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def run_scd_with_debug(source_df, dim_table_name, business_key_cols, attr_cols,
                      run_id = run_id or datetime.now().strftime("%Y%m%d_%H%M%S")):

    try:
        # Step 1: Compute hashes and detect changes
        source_df = source_df.withColumn("data_hash",
                                         md5(concat_ws("|", *attr_cols)))

        current_dim = spark.table(dim_table_name).filter(col("is_current") == True)
        current_dim = current_dim.withColumn("dim_hash",
                                             md5(concat_ws("|", *attr_cols)))

        # Join and classify
        joined = source_df.alias("s").join(
            current_dim.alias("d"),
            on=[col(f"s.{k}") == col(f"d.{k}") for k in business_key_cols],
            how="full_outer"
        )

        # New, changed, unchanged, deleted logic...
        changes = joined.withColumn("action",
                                    when(col("d.business_key").isNull(), "INSERT_NEW")
                                    .when(col("s.data_hash") != col("d.dim_hash"), "EXPIRE_AND_INSERT")
                                    .otherwise("NO_CHANGE"))

        # Step 2: Validation
        validation_issues = changes.filter(
            (col("action") == "EXPIRE_AND_INSERT") &
            (some_complex_condition_for_bad_data)
        )

        if validation_issues.count() > 0:

```

```

    # Quarantine
    validation_issues.write.mode("append").saveAsTable(f"quarantine_scd_
    logger.error(f"SCD validation issues found! Quarantined to table qua
    # Optionally raise or alert

# Step 3: Apply changes (MERGE or insert/expire)
# < your SCD Type 2 logic here (expire old + insert new) >

logger.info(f"SCD run {run_id} completed successfully.")

except Exception as e:
    logger.error(f"SCD pipeline failed for run {run_id}: {str(e)}")
    # Capture full context: source sample, dim state
    # Write error metadata table
    raise

```

Key Debug Features to Include

- **Run ID & Metadata Columns:** Add load_run_id, processed_timestamp, source_batch_id to all rows.
- **Quarantine Table:** Bad records go here with error codes (e.g., “OVERLAP”, “HASH_MISMATCH”, “NULL_KEY”).
- **Audit/History Log Table:** Record before/after states, row counts, and changed records count.
- **Delta Lake Time Travel:** VERSION AS OF or timestamps to inspect state before/after failure.
- **Error Handling Patterns:**
 - Try partial success (process good records, quarantine bad).
 - Circuit breakers for high error rates.
 - Detailed Spark UI / explain() logging for performance issues.

Monitoring Queries:


```
-- Quick health check
SELECT
    business_key,
    COUNT(DISTINCT effective_date) as versions,
    MAX(CASE WHEN is_current THEN 1 ELSE 0 END) as has_current
FROM dim_table
GROUP BY business_key
HAVING COUNT(*) > 1 AND MAX(CASE WHEN is_current THEN 1 ELSE 0 END) != 1;
```

Debugging Common Failures

1. **Silent Data Issues** → Add hash comparisons and row-level diffs.
2. **Pipeline Crashes** → Check logs, Spark UI (skew, OOM, shuffle), schema evolution.
3. **Incorrect History** → Use time travel + point-in-time queries.
4. **Performance** → Partition by business_key or date ranges; use Z-order/Bloom filters on Delta.

Best Practices for Robust SCD Pipelines

- Use declarative tools (e.g., dbt, Delta Live Tables Auto CDC) where possible to reduce custom code.
- Implement idempotency and re-runability.
- Version your dimension loads.
- Monitor data volume changes (alert on sudden spikes/drops).
- Document expected behavior for each dimension.
- Combine with CDC (Change Data Capture) from the source for efficiency.

Conclusion

Testing SCD isn't a one-time activity — integrate validations into your CI/CD and daily pipelines. A good debug pipeline turns scary failures into quick, actionable investigations. Start simple (manual SQL checks + logging), then automate and add quarantine/recovery.

By investing in these practices, you'll build trustworthy dimensional models that power reliable analytics and historical reporting.

Further Reading & Tools:

- SQLShack articles on SCD testing.
- Great Expectations or MonteCarlo for data quality.
- Delta Lake / Iceberg for built-in versioning.

[Data Analysis](#)[Big Data](#)[Pyspark](#)[Data Engineering](#)[Sql](#)

Written by Anurag Sharma

98 followers · 4 following

[Edit profile](#)

Data Engineering Specialist with 10+ exp. Passionate about optimizing pipelines, data lineage, and Spark performance and sharing insights to empower data pros!